

AI 210 NumPy Indexing and Slicing, Assignment *

Beaver-Edge AI Institute

1. Create an array `arr = np.arange(1,10)`.
 - (a) Print the first element.
 - (b) Print the last three elements.
2. Create an array with shape `(8,10)`.
 - (a) Print the row with index 2.
 - (b) Print the column with index 3.
 - (c) Use slicing to select rows with indices 2, 4, 6.
Print it out.
 - (d) Use fancy indexing to select rows with indices 2, 4, 6.
Print it out.
 - (e) Use slicing to select columns with indices 1, 4, 7.
Print it out.
 - (f) Use fancy indexing to select columns with indices 1, 4, 7.
Print it out.
 - (g) Use slicing to select entries with row indices 2, 4, 6, and column indices 1, 4, 7.
Print it out.
 - (h) Use fancy indexing to select entries with row indices 2, 4, 6, and column indices 1, 4, 7.
Print it out.
3. Create a 3×4 array `M = np.arange(12).reshape(3,4)`.
Slice out the middle two columns of all rows to get a subarray.
4. Write a function that uses fancy indexing to reverse the order of a 1-dim array.
For instance, if the original array is `[10, 20, 30, 40, 50]`. Then after applying your function, you should get a reversed array `[50, 40, 30, 20, 10]`.
5. Create `arr = np.array([5, 12, 3, 18, 7, 21])`.
 - (a) Use boolean indexing to extract all elements greater than 10.
 - (b) Use boolean indexing to extract all even numbers.

*Copyright © Beaver-Edge AI Institute. All Rights Reserved. No part of this document may be copied or reproduced without the written permission of Beaver-Edge AI Institute.

Copyright © Beaver-Edge AI Institute. All Rights Reserved. No part of this document may be copied or reproduced without the written permission of Beaver-Edge AI Institute.

- (c) Assign the value -1 to all elements in `arr` that are multiples of 3, and print the modified array.
6. Let `y` be a NumPy array with shape $(N,)$. Each entry is an integer from $\{0, 1, \dots, K-1\}$. Let `X` be a NumPy array with shape (N, K) . Each entry is a float. Create an array `z` with shape $(N,)$, such that `z[n] = X[n, y[n]]`.
7. Let `y` be a NumPy array with shape $(N,)$ (an array with a batch of labels). Each entry is an integer from $\{0, 1, \dots, K-1\}$. Let `X` be a NumPy array with shape $(N, 3, H, W)$ (an array with a batch of images). Create an array that consists of image data whose labels are equal to a target label value, say `k`. That is, this new array should include `X[n]` with `y[n] == k`, for all `n` in `range(N)`.
8. Let `X` be a NumPy array with shape (N, d) . Each entry is a float. Let `y = np.arange(N)`. We shuffle it by using
- ```
np.random.shuffle(y)
```
- Create an array that consists of rows in `X` whose row indices are positioned in top 80% of the shuffled `y`. The shape of this array is  $(\text{int}(0.3*N), d)$ .
9. (a) Create a  $3 \times 3$  random integer array `A` with values in  $[0, 15]$ . Use a fixed seed (e.g., 42).  
(b) Convert `A` to dtype `float32`.  
(c) Use boolean indexing to find which elements are multiples of 3. Print them.  
(d) Replace those multiples of 3 with 99.0 in `A` itself.  
(e) Print the final array.
10. (a) Create two 1-dim arrays:
- ```
x = np.array([10, 20, 30, 40, 50], dtype=np.float64)
y = np.array([ 5, 15, 25, 35, 45], dtype=np.float64)
```
- (b) Use fancy indexing to extract every other element from `x` (i.e., `[10, 30, 50]`).
(c) Use boolean indexing to extract all elements from `y` greater than 20.
(d) Compute the element-wise difference (`x - y`). Print the max and min of the resulting array.
(e) Define a custom function `g(z) = 2*z + 1` and apply it to the resulting difference array using `np.vectorize`.
11. Define an array `x` in the following way:
- ```
np.random.seed(42)
x = np.random.choice(np.arange(10,22), size = 12, replace = False).reshape(3,4)
np.random.seed()
```

Do the following tasks:

- (a) Print `x`.
- (b) Use `np.argsort` and fancy indexing to sort each column of `x`. Do not use `np.sort`.

12. Let `x` be an array with shape `(N,H,W,C)`. Let `y` and `idx` be arrays with shape `(N,)`.

Let `idx[n]` be the index of the  $n$ th smallest value in `y`.

We need to sort `x` along axis 0 in a way that after sorting, the new array has the following property:

`x_new[n] = x[idx[n]]` .

Use `np.argsort` and fancy indexing to sort each column of `x`. Do not use `np.sort`.

Beaver-Edge  
AI Institute